

BLACKROCK_PROJECT_5

1. Data Cleaning and Formatting

1.1 Remove/treat any special characters or non-numeric entries from financial fields.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df= pd.read_csv("C:/Users/BEAST/course 5 project/blackrock.csv")
df['TransactionAmount'] = pd.to_numeric(df['TransactionAmount'],
errors='coerce').fillna(0)
df['AccountBalance'] = pd.to_numeric(df['AccountBalance'],
errors='coerce').fillna(0)
df.isnull().sum()

TransactionID      0
CustomerID         0
AccountID          0
AccountType        0
TransactionType     0
Product            0
Firm               0
Region             0
Manager            0
TransactionDate     0
TransactionAmount   0
AccountBalance      0
RiskScore          0
CreditRating       0
TenureMonths       0
dtype: int64
```

1.2 Convert currency amounts into numerical format.

```
df['TransactionAmount'] = df['TransactionAmount'].replace('$', '',
regex=False).replace(',', '', regex=False).astype(float)
df['AccountBalance'] = df['AccountBalance'].replace('$', '',
regex=False).replace(',', '', regex=False).astype(float)
df['TransactionAmount'].head(5)
df['AccountBalance'].head(5)

0      2980.571037
1     27996.690070
2     81482.156180
3     39598.371940
```

```
4      111731.993700
Name: AccountBalance, dtype: float64
```

1.3 Validate and format date columns.

```
df['TransactionDate'] = pd.to_datetime(df
['TransactionDate'],dayfirst=True)
df ['TransactionDate'].head(5)

0      2023-07-17
1      2023-01-13
2      2023-09-28
3      2023-02-05
4      2023-11-19
Name: TransactionDate, dtype: datetime64[ns]
```

1.4 Ensure account types and transaction categories are standardized.

-This code cleans the AccountType column by removing extra spaces, converting values to text format, changing them into proper case, and correcting inconsistent names like wrong spellings. Finally, it displays the unique standardized account types.

```
# Remove extra spaces from column names
df.columns = df.columns.str.strip()

# Standardize AccountType
df['AccountType'] =
(df['AccountType'].astype(str).str.strip().str.title().replace({
    'Saving': 'Savings',
    'Current Acc': 'Current',
    'Currnt': 'Current'
}))
)
```

```
# Verify cleaned values
print("Unique Account Types:")
print(df['AccountType'].unique())
```

```
Unique Account Types:
['Credit' 'Current' 'Loan' 'Savings']
```

2. Descriptive Transactional Analysis

2.1 Calculate monthly and yearly summaries of total credits, debits, and net transaction volumn.¶

-This code first checks unique transaction types and converts TransactionDate into datetime format. It standardizes TransactionType values by removing spaces and formatting text properly. Then it separates transaction amounts into CreditAmount for deposits and

DebitAmount for withdrawals/payments. After that, it groups data monthly and yearly to calculate total credits, total debits, and net transaction volume (Credits – Debits). Finally, it prints the monthly and yearly summaries.

```
df['TransactionType'].unique()
df["TransactionDate"] = pd.to_datetime(df["TransactionDate"])
# Standardize transaction type text
df["TransactionType"] = df["TransactionType"].str.strip().str.title()

# Classify amounts
df["CreditAmount"] =
df["TransactionAmount"].where(df["TransactionType"] == "Deposit", 0)
df["DebitAmount"] =
df["TransactionAmount"].where(df["TransactionType"].isin(["Withdrawal",
, "Payment"]), 0)

# Monthly summary
monthly_summary = (
    df.groupby(df["TransactionDate"].dt.to_period("M"))
    .agg(
        TotalCredits=("CreditAmount", "sum"),
        TotalDebits=("DebitAmount", "sum")
    )
    .reset_index()
)

# Monthly Net volume
monthly_summary["NetTransactionVolume"] = (
    monthly_summary["TotalCredits"] - monthly_summary["TotalDebits"])

# Yearly summary
yearly_summary = (
    df.groupby(df["TransactionDate"].dt.to_period("Y"))
    .agg(
        TotalCredits=("CreditAmount", "sum"),
        TotalDebits=("DebitAmount", "sum")
    )
    .reset_index()
)

# Yearly net volume
yearly_summary["NetTransactionVolume"] =(
    yearly_summary["TotalCredits"] - yearly_summary["TotalDebits"])

print("Monthly Summary")
print(monthly_summary)
```

```
print("\nYearly Summary")
print(yearly_summary)
```

Monthly Summary

	TransactionDate	TotalCredits	TotalDebits	NetTransactionVolume
0	2023-01	4.532878e+05	1.329907e+06	-8.766195e+05
1	2023-02	5.565913e+05	1.674318e+06	-1.117726e+06
2	2023-03	4.055112e+05	1.397223e+06	-9.917113e+05
3	2023-04	3.289578e+05	1.273478e+06	-9.445206e+05
4	2023-05	9.335588e+05	1.320213e+06	-3.866543e+05
5	2023-06	6.760825e+05	5.318630e+05	1.442195e+05
6	2023-07	4.865498e+05	1.047057e+06	-5.605071e+05
7	2023-08	8.198514e+04	1.511240e+06	-1.429255e+06
8	2023-09	7.491474e+05	1.565614e+06	-8.164665e+05
9	2023-10	1.322052e+06	1.973337e+06	-6.512850e+05
10	2023-11	9.661783e+05	1.337323e+06	-3.711449e+05
11	2023-12	5.228622e+05	6.229963e+05	-1.001341e+05
12	2024-01	7.924528e+05	6.735780e+05	1.188748e+05
13	2024-02	6.244462e+05	1.482601e+06	-8.581550e+05
14	2024-03	7.652950e+05	1.920983e+06	-1.155688e+06
15	2024-04	3.763509e+05	8.109118e+05	-4.345608e+05
16	2024-05	3.007872e+05	8.523487e+05	-5.515616e+05
17	2024-06	4.029900e+05	1.123202e+06	-7.202123e+05

Yearly Summary

	TransactionDate	TotalCredits	TotalDebits	NetTransactionVolume
0	2023	7.482764e+06	1.558457e+07	-8.101805e+06
1	2024	3.262322e+06	6.863625e+06	-3.601303e+06

2.2 Plot trends in total credits vs. debits over time

-This code sets TransactionDate as the index for time-based analysis. It creates Credit and Debit columns by separating deposit amounts as credits and withdrawal, payment, or transfer amounts as debits. Then it resamples the data monthly and calculates total credits and debits for each month. Finally, it plots a line chart to compare monthly credits and debits over time.

```
#df.reset_index(inplace=True)

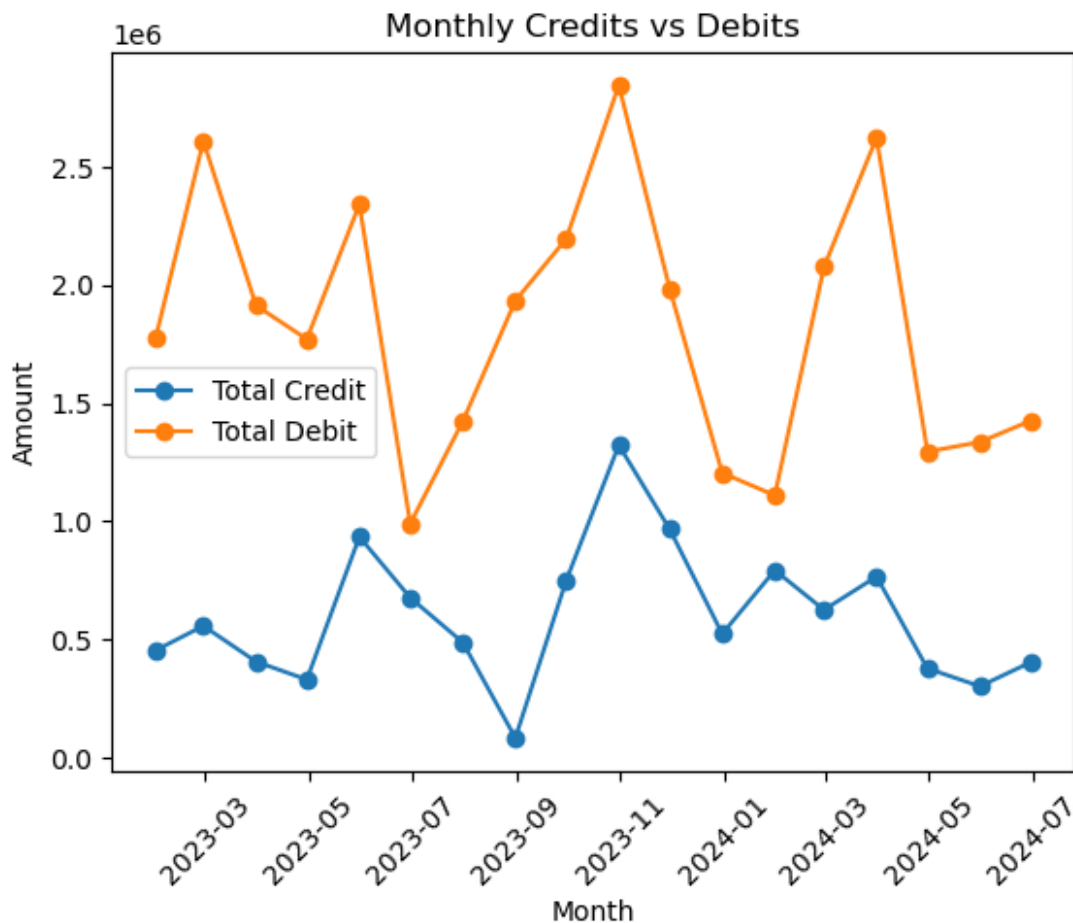
df.set_index('TransactionDate', inplace=True)

df['Credit'] = df['TransactionAmount'].where(df['TransactionType'] ==
'Deposit', 0)
df['Debit'] =
df['TransactionAmount'].where(df['TransactionType'].isin(["Withdrawal"
, "Payment", 'Transfer']), 0)

# Monthly aggregation
monthly_summary = df.resample('ME')[['Credit', 'Debit']].sum()

plt.figure()
```

```
plt.plot(monthly_summary.index, monthly_summary['Credit'], marker='o',
label='Total Credit')
plt.plot(monthly_summary.index, monthly_summary['Debit'], marker='o',
label='Total Debit')
plt.xlabel('Month')
plt.ylabel('Amount')
plt.title('Monthly Credits vs Debits')
plt.legend()
plt.xticks(rotation=45)
plt.show()
```



2.3 Identify top and bottom performing accounts based on net inflow¶

-This code first cleans the TransactionType column by removing spaces and standardizing text format. It then creates Inflow for deposit amounts and Outflow for withdrawal or payment amounts. Net inflow (NetFlow) is calculated as inflow minus outflow. After that, the data is grouped by AccountID to find total net flow for each account. Finally, it displays the top 5 performing accounts with highest net inflow and bottom 5 accounts with lowest net inflow.

```
#df.reset_index(inplace=True)
```

```

# Clean TransactionType (important)
df['TransactionType'] =
df['TransactionType'].astype(str).str.strip().str.title()

# Convert date (optional but good practice)
df['TransactionDate'] = pd.to_datetime(df['TransactionDate'],
dayfirst=True)

# Create inflow (+) and outflow (-)
df['Inflow'] = df['TransactionAmount'].where(df['TransactionType'] ==
"Deposit", 0)
df['Outflow'] =
df['TransactionAmount'].where(df['TransactionType'].isin(["Withdrawal"
, "Payment"]), 0 )

# Net inflow = inflow - outflow
df['NetFlow'] = df['Inflow'] - df['Outflow']

# Group by AccountID
account_summary = df.groupby('AccountID')
['NetFlow'].sum().reset_index()

# Top 5 performing accounts
top_accounts = account_summary.sort_values(by='NetFlow',
ascending=False).head(5)

# Bottom 5 performing accounts
bottom_accounts = account_summary.sort_values(by='NetFlow',
ascending=True).head(5)

print("Top 5 Performing Accounts")
print(top_accounts)
print("\nBottom 5 Performing Accounts")
print(bottom_accounts)

```

```

Top 5 Performing Accounts
   AccountID  NetFlow
170  ACC88286  284230.60079
24   ACC21719  274729.18572
102  ACC51200  201469.73244
43   ACC28295  199450.69292
169  ACC88252  183519.24882

```

```

Bottom 5 Performing Accounts
   AccountID  NetFlow
83   ACC45101 -395419.947830
66   ACC35419 -365901.203200
188  ACC97411 -365710.863533

```

```
138 ACC71938 -353666.209440
31 ACC24070 -338136.693810
```

2.4 Identify and flag accounts as dormant or inactive if there is a gap of two months or more between consecutive transactions.

-This code resets the index and converts TransactionDate into datetime format. It then sorts the data by AccountID and transaction date. Next, it calculates the number of days between consecutive transactions for each account. If the gap is 60 days or more, the account is marked as dormant; otherwise, it is active. Finally, it prints the list of dormant accounts and shows the records where dormancy occurred.

```
df.reset_index(inplace=True)

# Convert date column
df['TransactionDate'] = pd.to_datetime(df['TransactionDate'],
dayfirst=True)

# Sort data by AccountID and date
df = df.sort_values(by=['AccountID', 'TransactionDate'])

# Calculate gap (in days) between consecutive transactions per account
df['DaysGap'] = df.groupby('AccountID')
['TransactionDate'].diff().dt.days

# Flag dormant accounts (gap >= 60 days ~ 2 months)
df['DormantFlag'] = df['DaysGap'].apply(lambda x: 'Dormant' if x >= 60
else 'Active')

# Extract only dormant cases
dormant_accounts = df[df['DormantFlag'] == 'Dormant']

# Get unique account IDs that are dormant
dormant_account_list = dormant_accounts['AccountID'].unique()

# Print results
print("Dormant Accounts:")
print(dormant_account_list)

# Optional: view full records where dormancy occurred
print(dormant_accounts[['AccountID', 'TransactionDate', 'DaysGap']])
```

Dormant Accounts:

```
['ACC10117' 'ACC10996' 'ACC11062' 'ACC11188' 'ACC11285' 'ACC11837'
'ACC12334' 'ACC13357' 'ACC15228' 'ACC15671' 'ACC15925' 'ACC16241'
'ACC16664' 'ACC17688' 'ACC18057' 'ACC18140' 'ACC18177' 'ACC19156'
'ACC19178' 'ACC20297' 'ACC21264' 'ACC21429' 'ACC21719' 'ACC21878'
'ACC22036' 'ACC22255' 'ACC22799' 'ACC23736' 'ACC23985' 'ACC24070'
'ACC24508' 'ACC24880' 'ACC24981' 'ACC25811' 'ACC26026' 'ACC26940'
'ACC26956' 'ACC26973' 'ACC28154' 'ACC28295' 'ACC28305' 'ACC29007']
```

```
'ACC29231' 'ACC29356' 'ACC29646' 'ACC30146' 'ACC30787' 'ACC30852'
'ACC32212' 'ACC32627' 'ACC32890' 'ACC33287' 'ACC34119' 'ACC34431'
'ACC34568' 'ACC34821' 'ACC35163' 'ACC35419' 'ACC37688' 'ACC38559'
'ACC39161' 'ACC39482' 'ACC39500' 'ACC39529' 'ACC39544' 'ACC40939'
'ACC41829' 'ACC42467' 'ACC42710' 'ACC42903' 'ACC43309' 'ACC43771'
'ACC45101' 'ACC45521' 'ACC45907' 'ACC45951' 'ACC45968' 'ACC46655'
'ACC46953' 'ACC47099' 'ACC48303' 'ACC48501' 'ACC49140' 'ACC49180'
'ACC49364' 'ACC49395' 'ACC49422' 'ACC49774' 'ACC50439' 'ACC51009'
'ACC51200' 'ACC51593' 'ACC51971' 'ACC52131' 'ACC52650' 'ACC53466'
'ACC53865' 'ACC54589' 'ACC55331' 'ACC57516' 'ACC57597' 'ACC57700'
'ACC57872' 'ACC58078' 'ACC58667' 'ACC60432' 'ACC61827' 'ACC61926'
'ACC62446' 'ACC62809' 'ACC64022' 'ACC64393' 'ACC64785' 'ACC65144'
'ACC66086' 'ACC66190' 'ACC67701' 'ACC67713' 'ACC69323' 'ACC70314'
'ACC70460' 'ACC70741' 'ACC71388' 'ACC71426' 'ACC71938' 'ACC72197'
'ACC73104' 'ACC74631' 'ACC74656' 'ACC75675' 'ACC76549' 'ACC76597'
'ACC76699' 'ACC77592' 'ACC77638' 'ACC77773' 'ACC78178' 'ACC78581'
'ACC78589' 'ACC80131' 'ACC80442' 'ACC81631' 'ACC82298' 'ACC82381'
'ACC82926' 'ACC82947' 'ACC83005' 'ACC83581' 'ACC83848' 'ACC86784'
'ACC87006' 'ACC87602' 'ACC88252' 'ACC88286' 'ACC88449' 'ACC88516'
'ACC89098' 'ACC90887' 'ACC91220' 'ACC91723' 'ACC92104' 'ACC92360'
'ACC92558' 'ACC94203' 'ACC94907' 'ACC95774' 'ACC96277' 'ACC96868'
'ACC97225' 'ACC97411' 'ACC99409' 'ACC99549']
```

	AccountID	TransactionDate	DaysGap
724	ACC10117	2023-12-09	72.0
547	ACC10117	2024-04-25	104.0
144	ACC10996	2023-06-19	134.0
491	ACC10996	2023-10-06	109.0
18	ACC11062	2023-04-26	71.0
..	...	...	...
321	ACC97411	2023-09-26	184.0
29	ACC97411	2024-05-24	206.0
198	ACC99409	2023-12-21	125.0
398	ACC99409	2024-05-02	133.0
581	ACC99549	2023-12-30	312.0

[325 rows x 3 columns]

3. Customer Profile Building

3.1 Group accounts by activity levels: High, Medium, Low based on transaction frequency on your analysis and rubrics. Do not forget to mention the rubric in the headings.

-This code counts the total number of transactions for each AccountID. It then classifies accounts into activity levels based on transaction count: High for 10 or more transactions, Medium for 5 to 9 transactions, and Low for less than 5 transactions. After classification, the data is sorted in descending order of transaction count and displayed as a table.

```

# Count transactions per account
txn_count =
df.groupby('AccountID').size().reset_index(name='TransactionCount')

# Function to classify activity level
def classify_activity(count):
    if count >= 10:
        return 'High'
    elif count >= 5:
        return 'Medium'
    else:
        return 'Low'

# Apply classification
txn_count['ActivityLevel'] =
txn_count['TransactionCount'].apply(classify_activity)

# Sort for better view
txn_count = txn_count.sort_values(by='TransactionCount',
ascending=False)

# Display table
print(txn_count.to_string(index=False))

```

AccountID	TransactionCount	ActivityLevel
ACC41829	13	High
ACC29356	9	Medium
ACC11188	9	Medium
ACC28154	9	Medium
ACC62446	9	Medium
ACC16241	9	Medium
ACC52650	8	Medium
ACC43771	8	Medium
ACC34431	8	Medium
ACC57700	8	Medium
ACC70460	8	Medium
ACC97411	8	Medium
ACC96277	8	Medium
ACC92558	7	Medium
ACC61926	7	Medium
ACC46655	7	Medium
ACC81631	7	Medium
ACC25811	7	Medium
ACC45951	7	Medium
ACC83848	7	Medium
ACC76597	7	Medium
ACC45101	7	Medium
ACC76549	7	Medium
ACC77592	7	Medium
ACC53466	7	Medium

ACC78581	7	Medium
ACC40939	7	Medium
ACC45521	7	Medium
ACC29231	6	Medium
ACC26956	6	Medium
ACC10117	6	Medium
ACC30787	6	Medium
ACC64785	6	Medium
ACC34119	6	Medium
ACC61827	6	Medium
ACC53865	6	Medium
ACC45968	6	Medium
ACC35419	6	Medium
ACC71426	6	Medium
ACC77638	6	Medium
ACC22036	6	Medium
ACC11062	6	Medium
ACC11837	6	Medium
ACC95774	6	Medium
ACC92360	6	Medium
ACC11285	5	Medium
ACC49422	5	Medium
ACC94907	5	Medium
ACC42710	5	Medium
ACC15925	5	Medium
ACC88449	5	Medium
ACC88286	5	Medium
ACC83005	5	Medium
ACC21878	5	Medium
ACC64022	5	Medium
ACC88252	5	Medium
ACC48501	5	Medium
ACC67701	5	Medium
ACC24880	5	Medium
ACC80442	5	Medium
ACC74631	5	Medium
ACC72197	5	Medium
ACC24070	5	Medium
ACC29646	5	Medium
ACC71388	5	Medium
ACC51009	4	Low
ACC70741	4	Low
ACC51593	4	Low
ACC51200	4	Low
ACC65144	4	Low
ACC78178	4	Low
ACC91723	4	Low
ACC71938	4	Low
ACC10996	4	Low

ACC51971	4	Low
ACC89098	4	Low
ACC91220	4	Low
ACC66190	4	Low
ACC88516	4	Low
ACC54589	4	Low
ACC55331	4	Low
ACC55729	4	Low
ACC80131	4	Low
ACC60432	4	Low
ACC74656	4	Low
ACC87006	4	Low
ACC70314	4	Low
ACC82381	4	Low
ACC62809	4	Low
ACC64393	4	Low
ACC49180	4	Low
ACC99549	4	Low
ACC48303	4	Low
ACC18140	4	Low
ACC28305	4	Low
ACC28295	4	Low
ACC30146	4	Low
ACC26940	4	Low
ACC24981	4	Low
ACC32212	4	Low
ACC32627	4	Low
ACC23736	4	Low
ACC22799	4	Low
ACC34821	4	Low
ACC21719	4	Low
ACC39482	4	Low
ACC21264	4	Low
ACC29007	4	Low
ACC45907	4	Low
ACC13357	4	Low
ACC17688	4	Low
ACC16664	4	Low
ACC39544	4	Low
ACC94203	3	Low
ACC99409	3	Low
ACC12182	3	Low
ACC97225	3	Low
ACC75675	3	Low
ACC26026	3	Low
ACC76699	3	Low
ACC77773	3	Low
ACC78589	3	Low
ACC95164	3	Low

ACC18057	3	Low
ACC15671	3	Low
ACC22255	3	Low
ACC82298	3	Low
ACC82926	3	Low
ACC82947	3	Low
ACC83581	3	Low
ACC28612	3	Low
ACC87602	3	Low
ACC19178	3	Low
ACC19156	3	Low
ACC18177	3	Low
ACC90887	3	Low
ACC92104	3	Low
ACC49395	3	Low
ACC29396	3	Low
ACC42903	3	Low
ACC57597	3	Low
ACC57516	3	Low
ACC39500	3	Low
ACC34568	3	Low
ACC39529	3	Low
ACC32890	3	Low
ACC42467	3	Low
ACC57872	3	Low
ACC47099	3	Low
ACC30852	3	Low
ACC15228	2	Low
ACC96868	2	Low
ACC20297	2	Low
ACC49364	2	Low
ACC38559	2	Low
ACC39161	2	Low
ACC46953	2	Low
ACC12334	2	Low
ACC94242	2	Low
ACC52131	2	Low
ACC49774	2	Low
ACC50439	2	Low
ACC86784	2	Low
ACC15359	2	Low
ACC43309	2	Low
ACC37688	2	Low
ACC29477	2	Low
ACC23985	2	Low
ACC73104	2	Low
ACC49140	2	Low
ACC69323	2	Low
ACC67713	2	Low

ACC31539	2	Low
ACC66086	2	Low
ACC83269	2	Low
ACC24508	2	Low
ACC26973	2	Low
ACC31902	2	Low
ACC33287	2	Low
ACC35163	2	Low
ACC36079	2	Low
ACC58667	2	Low
ACC58078	2	Low
ACC21429	2	Low
ACC40952	1	Low
ACC78089	1	Low
ACC50817	1	Low
ACC25132	1	Low
ACC77533	1	Low
ACC28292	1	Low
ACC99117	1	Low
ACC65545	1	Low

3.2 Segment customers by average balance and transaction volume.

- This code groups the data by CustomerID and calculates each customer's average account balance and total number of transactions. The columns are renamed as AvgBalance and TxnCount. Median values of balance and transaction count are used as thresholds for segmentation. Customers are then classified into four groups: Premium Active, Premium Inactive, Regular Active, and Low Value based on their balance and activity level. Finally, the results are sorted and displayed.

```
# Calculate metrics per customer
customer_summary = df.groupby('CustomerID').agg({
    'AccountBalance': 'mean',
    'TransactionID': 'count'
}).reset_index()

# Rename columns
customer_summary.rename(columns={
    'AccountBalance': 'AvgBalance',
    'TransactionID': 'TxnCount'
}, inplace=True)

# Define thresholds (you can adjust)
balance_threshold = customer_summary['AvgBalance'].median()
txn_threshold = customer_summary['TxnCount'].median()

# Segmentation function
def segment(row):
    if row['AvgBalance'] >= balance_threshold and row['TxnCount'] >=
txn_threshold:
```

```

        return 'Premium Active'
    elif row['AvgBalance'] >= balance_threshold:
        return 'Premium Inactive'
    elif row['TxnCount'] >= txn_threshold:
        return 'Regular Active'
    else:
        return 'Low Value'

```

Apply segmentation

```
customer_summary['Segment'] = customer_summary.apply(segment, axis=1)
```

Sort for clarity

```
customer_summary = customer_summary.sort_values(by=['AvgBalance',
'TxnCount'], ascending=False)
```

Display result

```
print(customer_summary.to_string(index=False))
```

CustomerID	AvgBalance	TxnCount	Segment
CUST6947	137652.013500	1	Premium Inactive
CUST8344	131120.054500	1	Premium Inactive
CUST6735	116425.943257	3	Premium Inactive
CUST7345	114846.275627	3	Premium Inactive
CUST4346	114486.199587	3	Premium Inactive
CUST7395	112934.527150	2	Premium Inactive
CUST6526	108406.147097	4	Premium Active
CUST5320	107128.711898	6	Premium Active
CUST6391	101390.167900	1	Premium Inactive
CUST3109	100947.514670	4	Premium Active
CUST7481	100499.235467	4	Premium Active
CUST5428	99825.420003	3	Premium Inactive
CUST1888	98620.067583	3	Premium Inactive
CUST2357	98578.229917	7	Premium Active
CUST4855	98182.845137	3	Premium Inactive
CUST5664	97409.985772	4	Premium Active
CUST9962	95122.912120	1	Premium Inactive
CUST6837	93475.976380	4	Premium Active
CUST3131	92294.670120	2	Premium Inactive
CUST1547	92293.667688	6	Premium Active
CUST3059	91666.984465	4	Premium Active
CUST3725	90980.778360	7	Premium Active
CUST1497	90564.708385	4	Premium Active
CUST4438	90392.892004	9	Premium Active
CUST6350	90350.497025	4	Premium Active
CUST6330	90037.287977	3	Premium Inactive
CUST6038	88828.878772	5	Premium Active
CUST8250	88703.682603	3	Premium Inactive
CUST8390	88567.869393	11	Premium Active
CUST7493	88334.197798	4	Premium Active
CUST4780	87905.568873	7	Premium Active

CUST7459	87665.550803	3	Premium Inactive
CUST5253	87662.919621	9	Premium Active
CUST9066	87434.789445	4	Premium Active
CUST4126	86832.546587	3	Premium Inactive
CUST6028	86587.594158	6	Premium Active
CUST8874	86513.678688	5	Premium Active
CUST4360	86477.880765	6	Premium Active
CUST2457	86096.012450	2	Premium Inactive
CUST3778	85930.843993	3	Premium Inactive
CUST3609	85830.904460	5	Premium Active
CUST5705	85805.515614	5	Premium Active
CUST4769	85520.331097	6	Premium Active
CUST2942	85163.818244	5	Premium Active
CUST5105	85116.885562	4	Premium Active
CUST6776	85044.080995	2	Premium Inactive
CUST2694	84864.647213	3	Premium Inactive
CUST4884	84795.857435	4	Premium Active
CUST4241	84307.548772	5	Premium Active
CUST1747	83991.234560	5	Premium Active
CUST8529	83907.888822	4	Premium Active
CUST9038	83847.236106	10	Premium Active
CUST2427	83209.561123	4	Premium Active
CUST2915	82850.077710	5	Premium Active
CUST1609	82807.194783	6	Premium Active
CUST7793	81953.215391	7	Premium Active
CUST8266	81731.632325	2	Premium Inactive
CUST4794	81166.563550	4	Premium Active
CUST2944	81092.328050	1	Premium Inactive
CUST7028	80897.709680	3	Premium Inactive
CUST1376	80779.647074	5	Premium Active
CUST6319	79995.506083	3	Premium Inactive
CUST6029	79935.254490	9	Premium Active
CUST7388	79810.259330	2	Premium Inactive
CUST5500	79605.060397	3	Premium Inactive
CUST7651	79570.811356	7	Premium Active
CUST1042	79417.266317	6	Premium Active
CUST5174	79378.901268	5	Premium Active
CUST8660	79325.581940	3	Premium Inactive
CUST4763	79192.786180	2	Premium Inactive
CUST7855	79020.956278	6	Premium Active
CUST2188	79004.949287	3	Premium Inactive
CUST1738	78358.802540	4	Premium Active
CUST7730	77489.157608	5	Premium Active
CUST3153	76958.993526	7	Premium Active
CUST2612	76607.099448	5	Premium Active
CUST1223	76370.221527	7	Premium Active
CUST6549	75924.107853	3	Premium Inactive
CUST2578	75563.876932	9	Premium Active
CUST8772	75484.793463	3	Premium Inactive

CUST1555	74818.718354	5	Premium Active
CUST4461	74416.716192	6	Premium Active
CUST8318	74412.925453	3	Premium Inactive
CUST9528	73677.185965	3	Premium Inactive
CUST4595	73671.575266	5	Premium Active
CUST3602	73479.581588	6	Premium Active
CUST4258	73301.136218	6	Premium Active
CUST5121	73218.318234	7	Premium Active
CUST5916	72869.619923	6	Premium Active
CUST7255	72511.456285	4	Premium Active
CUST1768	72509.254770	6	Regular Active
CUST4684	72453.445554	9	Regular Active
CUST3006	71963.205374	7	Regular Active
CUST6082	71748.625818	5	Regular Active
CUST3015	71682.781510	2	Low Value
CUST3069	71271.668893	6	Regular Active
CUST1749	71205.460810	3	Low Value
CUST8765	71156.186792	5	Regular Active
CUST2412	71055.886920	3	Low Value
CUST1887	70599.965980	3	Low Value
CUST1962	70233.324889	8	Regular Active
CUST2349	70224.187390	5	Regular Active
CUST6937	69957.017527	6	Regular Active
CUST5351	69553.061830	6	Regular Active
CUST9169	69236.869402	5	Regular Active
CUST2871	69177.984562	5	Regular Active
CUST3187	68789.797741	8	Regular Active
CUST9129	68776.259713	6	Regular Active
CUST8416	67536.142336	5	Regular Active
CUST9683	67171.225589	7	Regular Active
CUST2699	67140.657123	4	Regular Active
CUST5920	67136.646548	5	Regular Active
CUST7098	67103.265467	7	Regular Active
CUST2443	66831.063394	5	Regular Active
CUST8028	66478.804269	7	Regular Active
CUST9439	66472.562097	3	Low Value
CUST8091	66267.279462	6	Regular Active
CUST4402	66051.385690	4	Regular Active
CUST8461	66050.179540	6	Regular Active
CUST7173	66029.297413	3	Low Value
CUST2805	65946.773817	4	Regular Active
CUST2541	65945.349395	2	Low Value
CUST2842	65874.391527	6	Regular Active
CUST9420	65765.300739	7	Regular Active
CUST1498	65424.298341	8	Regular Active
CUST1644	65243.098032	4	Regular Active
CUST5545	64930.961861	4	Regular Active
CUST1121	63924.640954	5	Regular Active
CUST5574	63119.641895	4	Regular Active

CUST7994	63116.262250	2	Low Value
CUST6292	62979.446228	6	Regular Active
CUST5610	62562.153933	6	Regular Active
CUST8279	62555.911077	3	Low Value
CUST3306	62157.206727	3	Low Value
CUST5558	62072.907250	2	Low Value
CUST5609	61795.787097	3	Low Value
CUST3762	61224.157804	5	Regular Active
CUST6525	60715.122427	4	Regular Active
CUST9666	60169.570125	2	Low Value
CUST9079	59969.473955	2	Low Value
CUST8288	59961.751980	3	Low Value
CUST9535	59420.573825	4	Regular Active
CUST4669	59379.662820	6	Regular Active
CUST7002	58146.066293	3	Low Value
CUST7887	57978.086593	4	Regular Active
CUST4625	57883.211673	4	Regular Active
CUST1776	57566.883870	1	Low Value
CUST9731	56981.186824	5	Regular Active
CUST1467	56689.613536	5	Regular Active
CUST6351	56675.138033	3	Low Value
CUST5385	55782.799878	5	Regular Active
CUST1840	55716.690107	3	Low Value
CUST8155	55149.231186	7	Regular Active
CUST1569	55028.817944	5	Regular Active
CUST3378	53570.918337	3	Low Value
CUST4584	53276.992580	3	Low Value
CUST1913	53031.790335	5	Regular Active
CUST2626	51929.887144	5	Regular Active
CUST1114	51474.223220	2	Low Value
CUST2716	51435.113382	4	Regular Active
CUST4373	50672.011875	5	Regular Active
CUST9564	50489.930038	6	Regular Active
CUST9216	49516.196833	6	Regular Active
CUST3315	48822.111133	7	Regular Active
CUST9754	48462.826090	4	Regular Active
CUST7933	44560.664548	3	Low Value
CUST9843	44101.312365	5	Regular Active
CUST3041	43496.780500	5	Regular Active
CUST5894	43470.241441	4	Regular Active
CUST9248	41667.098218	3	Low Value
CUST2067	41295.800465	2	Low Value
CUST9525	40295.986170	2	Low Value
CUST1189	39051.942297	3	Low Value
CUST3810	30028.381510	2	Low Value
CUST2109	29965.897650	1	Low Value
CUST5255	28882.864155	2	Low Value
CUST6565	28647.976127	4	Regular Active
CUST4003	27368.451306	4	Regular Active

CUST6210	21030.369426	4	Regular Active
CUST2464	4873.356091	1	Low Value

3.3 Create profiles for:

-High-net inflow accounts

-High-net inflow accounts

-High-frequency low-balance accounts

-Accounts with negative or near-zero balances

- This code reads the dataset and cleans the TransactionType column. It calculates Inflow for deposits, Outflow for withdrawals/payments, and NetFlow as inflow minus outflow. Then it groups data by AccountID to calculate total net flow, transaction count, and average account balance. Threshold values are set using quantiles and median values. Based on these thresholds, accounts are segmented into three groups: High Net Inflow Accounts, High Frequency Low Balance Accounts, and Low/Negative Balance Accounts. Finally, all segments are displayed.

```
import pandas as pd

df = pd.read_csv('blackrock.csv')

# --- Net Flow Calculation ---
df['TransactionType'] =
df['TransactionType'].astype(str).str.strip().str.title()

df['Inflow'] = df['TransactionAmount'].where(df['TransactionType'] ==
"Deposit", 0)
df['Outflow'] =
df['TransactionAmount'].where(df['TransactionType'].isin(["Withdrawal"
, "Payment"]), 0)
df['NetFlow'] = df['Inflow'] - df['Outflow']

account_summary = df.groupby('AccountID').agg({
    'NetFlow': 'sum',
    'TransactionID': 'count',
    'AccountBalance': 'mean'
}).reset_index()

# Thresholds
netflow_threshold = account_summary['NetFlow'].quantile(0.75)
txn_threshold = account_summary['TransactionID'].median()
balance_threshold = account_summary['AccountBalance'].median()

# Segments
high_net_inflow = account_summary[account_summary['NetFlow'] >=
netflow_threshold]

high_freq_low_bal = account_summary[
```

```

        (account_summary['TransactionID'] >= txn_threshold) &
        (account_summary['AccountBalance'] < balance_threshold)
    ]

low_balance_accounts = account_summary[
    account_summary['AccountBalance'] <= 5000    # near-zero threshold
]

# Print
print("High Net Inflow Accounts:\n", high_net_inflow)
print("\nHigh Frequency Low Balance Accounts:\n", high_freq_low_bal)
print("\nLow / Negative Balance Accounts:\n", low_balance_accounts)

```

High Net Inflow Accounts:

	AccountID	NetFlow	TransactionID	AccountBalance
6	ACC12182	80527.198540	3	103956.446880
9	ACC15228	46141.121310	2	130695.134600
10	ACC15359	69405.131370	2	78764.244350
18	ACC18177	94801.131690	3	67026.629560
23	ACC21429	54266.424140	2	66728.296900
24	ACC21719	274729.185720	4	72818.112908
27	ACC22255	60343.970740	3	68161.129063
33	ACC24880	82221.260590	5	69514.492810
36	ACC25811	12976.516798	7	79173.967039
39	ACC26956	40019.109429	6	80144.572470
41	ACC28154	10279.142964	9	69366.695512
43	ACC28295	199450.692920	4	125043.309138
44	ACC28305	2421.155330	4	71076.239332
50	ACC29477	57849.922720	2	84838.274600
51	ACC29646	26624.134440	5	106215.271614
56	ACC31902	103237.587600	2	65027.572310
58	ACC32627	125993.714440	4	93374.434050
60	ACC33287	43307.709902	2	59800.191530
63	ACC34568	10922.808390	3	25206.101480
64	ACC34821	35825.327080	4	23319.336055
68	ACC37688	29080.505480	2	89275.469620
71	ACC39482	61710.243160	4	72928.318290
75	ACC40939	83377.980683	7	94256.386354
85	ACC45907	99707.422688	4	62879.604810
96	ACC49395	81959.151390	3	60486.256503
97	ACC49422	64317.191830	5	92844.228626
102	ACC51200	201469.732440	4	72388.011257
103	ACC51593	80652.812730	4	78453.666545
116	ACC58078	7729.276320	2	66078.569850
124	ACC64393	70542.124380	4	55181.791659
130	ACC67701	47785.731750	5	75637.606770
133	ACC70314	95520.725840	4	68743.766100
140	ACC73104	12434.351310	2	100132.263085
141	ACC74631	70525.280750	5	64076.887205
144	ACC76549	4911.720110	7	84818.399456

145	ACC76597	13849.115040	7	56524.016080
147	ACC77533	69282.611860	1	144593.276200
151	ACC78089	69928.188380	1	104706.870800
158	ACC82298	44817.420360	3	72078.246633
163	ACC83269	180850.726890	2	93528.522860
164	ACC83581	51174.339160	3	68832.298013
167	ACC87006	89628.119090	4	87895.258590
169	ACC88252	183519.248820	5	86094.900381
170	ACC88286	284230.600790	5	71284.188672
171	ACC88449	134213.978110	5	65416.355338
174	ACC90887	57618.827680	3	86953.276387
187	ACC97225	42225.054470	3	53496.777053
189	ACC99117	92159.600750	1	106391.248100

High Frequency Low Balance Accounts:

	AccountID	NetFlow	TransactionID	AccountBalance
2	ACC11062	-96820.851390	6	57108.763888
4	ACC11285	-97497.309887	5	71205.645238
5	ACC11837	-280003.838417	6	69822.567937
12	ACC15925	-65739.103460	5	65404.556512
15	ACC17688	-59252.854200	4	70628.900610
17	ACC18140	-4319.779310	4	65493.644730
22	ACC21264	-46612.599990	4	49473.427420
29	ACC23736	-173747.545570	4	72555.684128
31	ACC24070	-338136.693810	5	53811.388618
33	ACC24880	82221.260590	5	69514.492810
41	ACC28154	10279.142964	9	69366.695512
44	ACC28305	2421.155330	4	71076.239332
46	ACC29007	-145368.855680	4	67857.441285
48	ACC29356	-26210.439168	9	72232.226249
52	ACC30146	-38350.201650	4	68019.630140
62	ACC34431	-160855.046270	8	55556.756888
64	ACC34821	35825.327080	4	23319.336055
66	ACC35419	-365901.203200	6	53390.330235
74	ACC39544	-78908.264010	4	60368.218282
77	ACC41829	-108828.282350	13	61628.656517
79	ACC42710	-146690.644470	5	63115.146550
82	ACC43771	-161157.988220	8	59242.759106
83	ACC45101	-395419.947830	7	65679.954558
84	ACC45521	-169342.206420	7	49875.368013
85	ACC45907	99707.422688	4	62879.604810
86	ACC45951	-168822.128270	7	72398.409783
92	ACC48501	-141368.090430	5	53845.585612
94	ACC49180	-110701.303270	4	66336.088125
102	ACC51200	201469.732440	4	72388.011257
104	ACC51971	-130945.408250	4	65905.262018
107	ACC53466	-314267.167030	7	71688.032449
108	ACC53865	-8089.641240	6	70396.222055
109	ACC54589	-54100.895640	4	57435.458890

118	ACC60432	-65042.789220	4	53330.553720
119	ACC61827	-129715.308219	6	47266.088053
121	ACC62446	-140605.456180	9	65782.376197
124	ACC64393	70542.124380	4	55181.791659
126	ACC65144	-9098.350710	4	51640.779318
133	ACC70314	95520.725840	4	68743.766100
138	ACC71938	-353666.209440	4	54440.037162
139	ACC72197	-217989.567518	5	63712.879012
141	ACC74631	70525.280750	5	64076.887205
145	ACC76597	13849.115040	7	56524.016080
148	ACC77592	-262254.424839	7	63568.362033
149	ACC77638	-46329.366170	6	54284.430696
153	ACC78581	-332353.939521	7	60382.951011
162	ACC83005	-155575.032640	5	71705.969450
165	ACC83848	-229694.629630	7	65004.126281
170	ACC88286	284230.600790	5	71284.188672
171	ACC88449	134213.978110	5	65416.355338
172	ACC88516	-130426.032320	4	41461.224302
178	ACC92360	-108827.973440	6	56191.522772
179	ACC92558	-62143.404250	7	64820.008179
182	ACC94907	-211626.066760	5	65256.809974
185	ACC96277	-78044.651736	8	53257.609105
191	ACC99549	-121474.083870	4	44425.552420

Low / Negative Balance Accounts:

Empty DataFrame

Columns: [AccountID, NetFlow, TransactionID, AccountBalance]

Index: []

4. Financial Risk Identification

4.1 Track accounts with frequent large withdrawals or overdrafts.

- This code first cleans the TransactionType column and filters only withdrawal transactions. It sets a threshold using the top 20% highest withdrawal amounts and flags them as large withdrawals. Then, it counts how many large withdrawals each account has made and selects accounts with 3 or more such transactions. It also identifies overdraft accounts where the balance is negative. Finally, both risky groups are merged to create a list of accounts with frequent large withdrawals or overdrafts, and the result is displayed.

```
# Clean TransactionType
df['TransactionType'] =
df['TransactionType'].astype(str).str.strip().str.title()

# --- Identify Withdrawals ---
withdrawals = df[df['TransactionType'] == 'Withdrawal']

# Threshold for large withdrawals (top 10%)
large_threshold = withdrawals['TransactionAmount'].quantile(0.80)
```

```

# Flag large withdrawals
withdrawals['LargeWithdrawal'] = withdrawals['TransactionAmount'] >=
large_threshold

# Count frequent large withdrawals per account
freq_large =
withdrawals[withdrawals['LargeWithdrawal']].groupby('AccountID').size(
).reset_index(name='LargeTxnCount')

# Filter frequent (>=3 times)
freq_large_accounts = freq_large[freq_large['LargeTxnCount'] >= 3]

# --- Identify Overdraft Accounts ---
overdraft_accounts = df[df['AccountBalance'] < 0]
['AccountID'].unique()

# Convert to DataFrame
overdraft_df = pd.DataFrame(overdraft_accounts, columns=['AccountID'])

# --- Merge both risks ---
risk_accounts = pd.merge(freq_large_accounts, overdraft_df,
on='AccountID', how='outer')

# Fill missing values
risk_accounts['LargeTxnCount'] =
risk_accounts['LargeTxnCount'].fillna(0)

# Print result
print("Accounts with Frequent Large Withdrawals or Overdrafts:\n")
print(risk_accounts.to_string(index=False))

```

Accounts with Frequent Large Withdrawals or Overdrafts:

AccountID	LargeTxnCount
ACC11062	0.0
ACC15671	0.0
ACC22036	0.0
ACC30787	0.0
ACC34821	0.0
ACC39529	0.0
ACC45101	0.0
ACC45521	0.0
ACC46953	0.0
ACC64022	0.0
ACC74631	0.0
ACC78589	0.0
ACC87602	0.0
ACC88252	0.0
ACC88516	0.0

```
ACC94907      0.0
ACC97225      0.0
```

C:\Users\BEAST\AppData\Local\Temp\ipykernel_12908\2146960010.py:11:

SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.

Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation:

https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
withdrawals['LargeWithdrawal'] = withdrawals['TransactionAmount'] >=
large_threshold
```

3.2 Calculate balance volatility using standard deviation or coefficient of variation.

- This code groups the data by AccountID and calculates the average account balance and standard deviation of balance for each account. It then computes the Coefficient of Variation (CV) by dividing standard deviation by average balance, which measures balance volatility. Missing values caused by division errors are replaced with zero. Accounts are sorted by highest CV, and each account is classified as High, Medium, or Low Volatility based on CV values. Finally, the volatility report is displayed.

```
# Group by AccountID and calculate metrics
volatility = df.groupby('AccountID')['AccountBalance'].agg(['mean',
'std']).reset_index()

# Rename columns
volatility.rename(columns={'mean': 'AvgBalance', 'std': 'StdDev'},
inplace=True)

# Calculate Coefficient of Variation (CV)
volatility['CV'] = volatility['StdDev'] / volatility['AvgBalance']

# Handle division by zero (important)
volatility['CV'] = volatility['CV'].fillna(0)

# Sort by highest volatility
volatility = volatility.sort_values(by='CV', ascending=False)

# Display result
#print(volatility.to_string(index=False))

def classify_volatility(cv):
    if cv > 0.5:
        return 'High Volatility'
    elif cv > 0.2:
        return 'Medium Volatility'
    else:
        return 'Low Volatility'
```

```
volatility['VolatilityLevel'] =
volatility['CV'].apply(classify_volatility)

print(volatility.to_string(index=False))
```

AccountID	AvgBalance	StdDev	CV	VolatilityLevel
ACC78589	34272.665816	86423.147835	2.521635	High Volatility
ACC46953	35656.159735	88564.262571	2.483842	High Volatility
ACC34821	23319.336055	48974.249807	2.100156	High Volatility
ACC39529	62774.934093	91165.113157	1.452253	High Volatility
ACC15671	27077.566847	30995.763211	1.144703	High Volatility
ACC87602	41022.300481	46348.764739	1.129843	High Volatility
ACC38559	66090.737960	70149.365923	1.061410	High Volatility
ACC28612	42258.706897	44658.868241	1.056797	High Volatility
ACC97225	53496.777053	51171.428405	0.956533	High Volatility
ACC76699	46970.955587	44554.662869	0.948558	High Volatility
ACC88516	41461.224302	37412.644201	0.902353	High Volatility
ACC20297	80846.693440	70164.033043	0.867865	High Volatility
ACC75675	57655.972321	49954.686637	0.866427	High Volatility
ACC64393	55181.791659	46765.585665	0.847482	High Volatility
ACC45521	49875.368013	42175.165828	0.845611	High Volatility
ACC28305	71076.239332	59418.129844	0.835977	High Volatility
ACC99409	55933.330168	46489.352762	0.831157	High Volatility
ACC94242	66269.623970	53554.585160	0.808132	High Volatility
ACC49140	83313.414930	64670.361092	0.776230	High Volatility
ACC22036	74872.558018	57748.822230	0.771295	High Volatility
ACC30146	68019.630140	52382.746486	0.770112	High Volatility
ACC39500	64181.416867	49007.262673	0.763574	High Volatility
ACC94203	67858.855403	51350.311192	0.756722	High Volatility
ACC88252	86094.900381	65054.864364	0.755618	High Volatility
ACC57872	46600.817439	34660.931923	0.743784	High Volatility
ACC95164	94768.054193	70375.125823	0.742604	High Volatility
ACC96868	90122.875785	66366.896010	0.736405	High Volatility
ACC92360	56191.522772	40505.480482	0.720847	High Volatility
ACC64022	75321.458938	53560.925559	0.711098	High Volatility
ACC39544	60368.218282	42809.414927	0.709138	High Volatility
ACC11062	57108.763888	40017.803232	0.700730	High Volatility
ACC77638	54284.430696	37499.689534	0.690800	High Volatility
ACC24070	53811.388618	36563.451377	0.679474	High Volatility
ACC21719	72818.112908	49213.497040	0.675841	High Volatility
ACC29477	84838.274600	57288.201836	0.675264	High Volatility
ACC45101	65679.954558	43417.303411	0.661043	High Volatility
ACC74631	64076.887205	42311.680971	0.660327	High Volatility
ACC54589	57435.458890	37156.119193	0.646920	High Volatility
ACC41829	61628.656517	39370.220877	0.638830	High Volatility
ACC94907	65256.809974	41288.149776	0.632703	High Volatility
ACC34431	55556.756888	34505.958913	0.621094	High Volatility
ACC26973	50183.235590	29473.812081	0.587324	High Volatility
ACC77592	63568.362033	37124.591651	0.584011	High Volatility
ACC83005	71705.969450	41804.713773	0.583002	High Volatility

ACC39161	86606.138565	50062.751874	0.578051	High	Volatility
ACC66190	82600.275190	46910.654939	0.567924	High	Volatility
ACC78581	60382.951011	34278.423693	0.567684	High	Volatility
ACC30787	76017.615717	43086.301089	0.566794	High	Volatility
ACC49364	70660.634835	40040.923745	0.566665	High	Volatility
ACC42710	63115.146550	35151.664571	0.556945	High	Volatility
ACC45951	72398.409783	40177.200417	0.554946	High	Volatility
ACC21264	49473.427420	27299.719843	0.551806	High	Volatility
ACC76549	84818.399456	46224.593643	0.544983	High	Volatility
ACC71938	54440.037162	29452.521538	0.541008	High	Volatility
ACC70460	73479.799788	39417.103944	0.536435	High	Volatility
ACC71426	81387.863613	43090.379183	0.529445	High	Volatility
ACC80131	82022.622885	42401.876761	0.516953	High	Volatility
ACC83848	65004.126281	33532.189928	0.515847	High	Volatility
ACC89098	73531.283570	37679.586561	0.512429	High	Volatility
ACC49422	92844.228626	47234.919735	0.508755	High	Volatility
ACC57700	72599.169237	36555.676527	0.503527	High	Volatility
ACC11285	71205.645238	35772.557175	0.502384	High	Volatility
ACC18177	67026.629560	33632.516562	0.501778	High	Volatility
ACC29396	51895.393960	25792.745992	0.497014	Medium	Volatility
ACC29646	106215.271614	52487.976280	0.494166	Medium	Volatility
ACC60432	53330.553720	26344.858659	0.493992	Medium	Volatility
ACC47099	89739.494267	43644.333465	0.486345	Medium	Volatility
ACC67701	75637.606770	36061.660226	0.476769	Medium	Volatility
ACC51200	72388.011257	33957.070488	0.469098	Medium	Volatility
ACC66086	125247.065790	58652.210406	0.468292	Medium	Volatility
ACC99549	44425.552420	20655.287474	0.464942	Medium	Volatility
ACC52650	78290.971593	36336.662098	0.464123	Medium	Volatility
ACC22799	80221.374125	36400.430881	0.453750	Medium	Volatility
ACC83581	68832.298013	30894.608297	0.448839	Medium	Volatility
ACC23736	72555.684128	32334.049854	0.445645	Medium	Volatility
ACC49774	91174.333570	40360.777452	0.442677	Medium	Volatility
ACC35419	53390.330235	23562.363963	0.441323	Medium	Volatility
ACC92558	64820.008179	28582.938612	0.440959	Medium	Volatility
ACC26956	80144.572470	34387.081079	0.429063	Medium	Volatility
ACC91723	79332.406083	33882.689444	0.427098	Medium	Volatility
ACC40939	94256.386354	39353.172785	0.417512	Medium	Volatility
ACC16241	84908.403037	35341.027502	0.416225	Medium	Volatility
ACC48303	88500.847185	36730.240299	0.415027	Medium	Volatility
ACC53865	70396.222055	28989.637711	0.411807	Medium	Volatility
ACC71388	75431.137826	30932.196460	0.410072	Medium	Volatility
ACC26940	85937.699960	35186.324219	0.409440	Medium	Volatility
ACC95774	81714.809018	33090.001570	0.404945	Medium	Volatility
ACC26026	65926.832903	26656.838209	0.404340	Medium	Volatility
ACC48501	53845.585612	21712.598945	0.403238	Medium	Volatility
ACC51971	65905.262018	26133.404709	0.396530	Medium	Volatility
ACC23985	90980.532390	35843.347687	0.393967	Medium	Volatility
ACC61827	47266.088053	18548.467956	0.392427	Medium	Volatility
ACC34119	83689.891720	32216.345337	0.384949	Medium	Volatility

ACC15925	65404.556512	24933.890052	0.381226	Medium	Volatility
ACC10117	81106.206188	30732.940500	0.378922	Medium	Volatility
ACC67713	116658.764295	43374.000251	0.371802	Medium	Volatility
ACC24880	69514.492810	25594.575722	0.368190	Medium	Volatility
ACC32627	93374.434050	34358.143099	0.367961	Medium	Volatility
ACC64785	91249.052553	33564.676915	0.367836	Medium	Volatility
ACC76597	56524.016080	20564.285254	0.363815	Medium	Volatility
ACC53466	71688.032449	26000.690531	0.362692	Medium	Volatility
ACC88449	65416.355338	23690.386433	0.362148	Medium	Volatility
ACC55331	73221.782365	25924.698246	0.354057	Medium	Volatility
ACC17688	70628.900610	24845.437316	0.351774	Medium	Volatility
ACC96277	53257.609105	18268.177617	0.343015	Medium	Volatility
ACC88286	71284.188672	24294.836479	0.340817	Medium	Volatility
ACC65144	51640.779318	17574.491194	0.340322	Medium	Volatility
ACC97411	89407.828831	30357.592275	0.339541	Medium	Volatility
ACC73104	100132.263085	33955.796994	0.339109	Medium	Volatility
ACC92104	100388.641920	34038.532459	0.339068	Medium	Volatility
ACC18140	65493.644730	22117.355592	0.337702	Medium	Volatility
ACC82381	81518.370007	26360.859147	0.323373	Medium	Volatility
ACC82298	72078.246633	23016.797331	0.319331	Medium	Volatility
ACC32890	50764.669013	16197.636605	0.319073	Medium	Volatility
ACC91220	89574.648937	28549.916580	0.318728	Medium	Volatility
ACC11188	80235.151167	25420.451644	0.316824	Medium	Volatility
ACC21429	66728.296900	20877.383293	0.312872	Medium	Volatility
ACC29231	102089.718303	31485.630851	0.308411	Medium	Volatility
ACC49395	60486.256503	18318.585929	0.302855	Medium	Volatility
ACC11837	69822.567937	21045.250870	0.301410	Medium	Volatility
ACC72197	63712.879012	19121.073456	0.300113	Medium	Volatility
ACC43771	59242.759106	17410.641688	0.293886	Medium	Volatility
ACC46655	79387.951184	22948.230586	0.289064	Medium	Volatility
ACC39482	72928.318290	20636.029492	0.282963	Medium	Volatility
ACC45907	62879.604810	17754.759170	0.282361	Medium	Volatility
ACC34568	25206.101480	7117.198745	0.282360	Medium	Volatility
ACC62446	65782.376197	18396.616973	0.279659	Medium	Volatility
ACC55729	74942.701735	20836.121305	0.278027	Medium	Volatility
ACC42467	70388.203053	19227.973079	0.273170	Medium	Volatility
ACC80442	87935.567978	23909.941064	0.271903	Medium	Volatility
ACC62809	78570.384865	21029.831135	0.267656	Medium	Volatility
ACC25811	79173.967039	21104.523800	0.266559	Medium	Volatility
ACC31539	51000.487165	13496.241952	0.264630	Medium	Volatility
ACC52131	75615.794590	19747.988934	0.261162	Medium	Volatility
ACC57516	69445.638753	18087.856914	0.260461	Medium	Volatility
ACC15228	130695.134600	33896.592195	0.259356	Medium	Volatility
ACC61926	82775.154450	21258.793491	0.256826	Medium	Volatility
ACC57597	65281.785570	16700.418779	0.255820	Medium	Volatility
ACC28154	69366.695512	17650.566417	0.254453	Medium	Volatility
ACC51009	79977.125935	20192.003920	0.252472	Medium	Volatility
ACC70314	68743.766100	17091.815152	0.248631	Medium	Volatility
ACC82947	48248.921820	11725.937905	0.243030	Medium	Volatility

ACC28295	125043.309138	29893.862616	0.239068	Medium	Volatility
ACC83269	93528.522860	21744.154842	0.232487	Medium	Volatility
ACC81631	86836.523123	19829.119003	0.228350	Medium	Volatility
ACC30852	72654.314937	16521.682977	0.227401	Medium	Volatility
ACC29356	72232.226249	16379.394091	0.226760	Medium	Volatility
ACC77773	104327.081090	23307.166508	0.223405	Medium	Volatility
ACC24981	83432.023110	18543.034508	0.222253	Medium	Volatility
ACC24508	97629.128980	21694.200407	0.222210	Medium	Volatility
ACC16664	98622.948550	21808.981546	0.221135	Medium	Volatility
ACC45968	75266.253778	16080.988132	0.213655	Medium	Volatility
ACC32212	98212.607462	20962.340988	0.213438	Medium	Volatility
ACC82926	82118.478787	17080.060903	0.207993	Medium	Volatility
ACC12334	71687.818925	14806.560499	0.206542	Medium	Volatility
ACC78178	73142.267595	15023.594800	0.205402	Medium	Volatility
ACC21878	96625.142774	19410.610225	0.200886	Medium	Volatility
ACC58667	83319.454240	16735.168443	0.200855	Medium	Volatility
ACC70741	92668.123342	18226.496556	0.196686	Low	Volatility
ACC35163	62313.224095	12253.234809	0.196639	Low	Volatility
ACC29007	67857.441285	13258.866819	0.195393	Low	Volatility
ACC69323	100023.436420	18259.781583	0.182555	Low	Volatility
ACC87006	87895.258590	15099.851103	0.171794	Low	Volatility
ACC51593	78453.666545	13172.031614	0.167896	Low	Volatility
ACC18057	102741.705097	16558.684186	0.161168	Low	Volatility
ACC37688	89275.469620	14174.212476	0.158769	Low	Volatility
ACC58078	66078.569850	9832.678211	0.148803	Low	Volatility
ACC22255	68161.129063	9910.285728	0.145395	Low	Volatility
ACC13357	82050.047215	11887.323984	0.144879	Low	Volatility
ACC49180	66336.088125	9502.483048	0.143248	Low	Volatility
ACC19156	94271.336230	12949.336108	0.137362	Low	Volatility
ACC42903	69670.986383	8957.113963	0.128563	Low	Volatility
ACC90887	86953.276387	11063.272602	0.127232	Low	Volatility
ACC10996	101733.898198	12477.429689	0.122648	Low	Volatility
ACC43309	79723.972460	9534.333469	0.119592	Low	Volatility
ACC74656	79851.939380	8846.045171	0.110781	Low	Volatility
ACC12182	103956.446880	11495.224452	0.110577	Low	Volatility
ACC36079	101690.753005	10253.137557	0.100827	Low	Volatility
ACC19178	113647.165867	6864.222596	0.060399	Low	Volatility
ACC86784	77901.376725	3331.652613	0.042768	Low	Volatility
ACC50439	38766.962880	791.574703	0.020419	Low	Volatility
ACC33287	59800.191530	897.088993	0.015001	Low	Volatility
ACC31902	65027.572310	798.516992	0.012280	Low	Volatility
ACC15359	78764.244350	839.916383	0.010664	Low	Volatility
ACC28292	74043.855810	NaN	0.000000	Low	Volatility
ACC50817	124809.189200	NaN	0.000000	Low	Volatility
ACC77533	144593.276200	NaN	0.000000	Low	Volatility
ACC78089	104706.870800	NaN	0.000000	Low	Volatility
ACC25132	152303.899800	NaN	0.000000	Low	Volatility
ACC40952	55394.828610	NaN	0.000000	Low	Volatility

ACC99117	106391.248100	NaN	0.000000	Low Volatility
ACC65545	171767.657800	NaN	0.000000	Low Volatility

4.3 Use IQR or z-score methods to detect anomalies.

- This code first cleans the TransactionType column and detects transaction amount anomalies using the IQR method. It then identifies large transactions using the top 10% transaction amounts and counts them for each customer. Customers with negative balances are marked as overdraft cases. Next, balance volatility is measured using the Coefficient of Variation (CV). All indicators are merged into one summary table, and customers are flagged as suspicious if they have frequent large transactions, overdrafts, or high volatility. Finally, the suspicious customers list is displayed.

```
# Clean data
df['TransactionType'] =
df['TransactionType'].astype(str).str.strip().str.title()

# -----
# 1. IQR Anomaly Detection
Q1 = df['TransactionAmount'].quantile(0.25)
Q3 = df['TransactionAmount'].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

df['Anomaly'] = ((df['TransactionAmount'] < lower) |
                 (df['TransactionAmount'] > upper))

# -----
# 2. Large Withdrawals
withdrawals = df[df['TransactionType'] != 'Credit']
threshold = withdrawals['TransactionAmount'].quantile(0.90)

withdrawals['LargeTxn'] = withdrawals['TransactionAmount'] >=
threshold

large_txn_count =
withdrawals[withdrawals['LargeTxn']].groupby('CustomerID').size().reset_index(name='LargeTxnCount')

# -----
# 3. Overdraft
overdraft = df[df['AccountBalance'] < 0]['CustomerID'].unique()

# -----
# 4. Volatility (CV)
volatility = df.groupby('CustomerID')
['AccountBalance'].agg(['mean', 'std']).reset_index()
volatility['CV'] = volatility['std'] / volatility['mean']
```

```

# -----
# Merge all indicators
summary = df.groupby('CustomerID').size().reset_index(name='TxnCount')

summary = summary.merge(large_txn_count, on='CustomerID', how='left')
summary = summary.merge(volatility[['CustomerID', 'CV']],
on='CustomerID', how='left')

summary['LargeTxnCount'] = summary['LargeTxnCount'].fillna(0)

# Flag suspicious customers
summary['Suspicious'] = (
    (summary['LargeTxnCount'] >= 3) |
    (summary['CustomerID'].isin(overdraft)) |
    (summary['CV'] > 0.5)
)
# Extract suspicious customers
suspicious_customers = summary[summary['Suspicious'] == True]

# Display
print("Suspicious Customers:\n")
print(suspicious_customers.to_string(index=False))

```

Suspicious Customers:

CustomerID	TxnCount	LargeTxnCount	CV	Suspicious
CUST1042	6	1.0	0.566128	True
CUST1114	2	0.0	0.987340	True
CUST1467	5	1.0	0.899920	True
CUST1555	5	0.0	0.539309	True
CUST1569	5	0.0	0.511648	True
CUST1644	4	1.0	1.066275	True
CUST1738	4	0.0	0.540951	True
CUST1913	5	0.0	0.803556	True
CUST1962	8	2.0	0.588642	True
CUST2067	2	0.0	0.785164	True
CUST2188	3	0.0	0.563943	True
CUST2412	3	1.0	0.817460	True
CUST2578	9	0.0	0.757872	True
CUST2626	5	0.0	0.526893	True
CUST2694	3	1.0	0.825217	True
CUST3069	6	0.0	0.558832	True
CUST3153	7	0.0	0.608230	True
CUST3315	7	2.0	0.754844	True
CUST3602	6	3.0	0.280482	True
CUST3810	2	1.0	0.605133	True
CUST4003	4	0.0	1.193350	True
CUST4126	3	0.0	0.739785	True
CUST4346	3	0.0	0.543854	True
CUST4360	6	1.0	0.673988	True

CUST4373	5	1.0	0.831068	True
CUST4584	3	0.0	0.529137	True
CUST4684	9	2.0	0.594707	True
CUST4769	6	1.0	0.698288	True
CUST5174	5	0.0	0.679998	True
CUST5545	4	0.0	0.779460	True
CUST5610	6	1.0	0.864795	True
CUST5705	5	0.0	0.551221	True
CUST5894	4	0.0	0.845125	True
CUST5916	6	0.0	0.587104	True
CUST6038	5	1.0	0.545645	True
CUST6210	4	0.0	1.646892	True
CUST6319	3	0.0	0.539343	True
CUST6350	4	0.0	0.621406	True
CUST6549	3	0.0	0.567960	True
CUST6565	4	1.0	1.525411	True
CUST7493	4	0.0	0.506760	True
CUST7793	7	0.0	0.615699	True
CUST7933	3	0.0	0.870658	True
CUST8279	3	0.0	0.578759	True
CUST8416	5	1.0	0.841165	True
CUST8660	3	0.0	0.774203	True
CUST8772	3	0.0	0.520629	True
CUST9216	6	0.0	0.699997	True
CUST9248	3	1.0	1.578156	True
CUST9528	3	0.0	0.811485	True
CUST9535	4	2.0	0.731887	True
CUST9564	6	1.0	0.684885	True
CUST9666	2	0.0	0.781755	True
CUST9683	7	1.0	0.695660	True
CUST9731	5	0.0	0.956280	True
CUST9843	5	1.0	0.808967	True

5. Visualisation¶

Baasic info

- This code imports the pandas and matplotlib libraries, then reads the blackrock.csv dataset into a DataFrame. df.head() displays the first five rows of the dataset, df.info() shows column names, data types, and missing values, and df.describe() provides statistical summary such as count, mean, standard deviation, minimum, and maximum values for numerical columns.

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv('blackrock.csv')

print(df.head())
print(df.info())
print(df.describe())
```

	TransactionID	CustomerID	AccountID	AccountType	TransactionType	\
0	109	CUST1913	ACC64393	Credit	Withdrawal	
1	19	CUST1569	ACC66190	Current	Payment	
2	14	CUST5558	ACC71426	Current	Withdrawal	
3	107	CUST4241	ACC49422	Loan	Deposit	
4	7	CUST2578	ACC88252	Loan	Deposit	

	Product	Firm	Region	Manager	TransactionDate	\
0	Home Loan	Firm A	West	Manager 1	17-07-2023	
1	Personal Loan	Firm C	South	Manager 4	13-01-2023	
2	Mutual Fund	Firm C	North	Manager 2	28-09-2023	
3	Credit Card	Firm D	Central	Manager 1	05-02-2023	
4	Savings Account	Firm B	East	Manager 4	19-11-2023	

	TransactionAmount	AccountBalance	RiskScore	CreditRating
0	47387.11457	2980.571037	0.780701	784
170				
1	55806.54015	27996.690070	0.299268	336
197				
2	51080.47837	81482.156180	0.388445	712
95				
3	70472.70472	39598.371940	0.561809	414
168				
4	29830.48767	111731.993700	0.655635	391
20				

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 800 entries, 0 to 799
Data columns (total 15 columns):

#	Column	Non-Null Count	Dtype
---	-----	-----	-----
0	TransactionID	800 non-null	int64
1	CustomerID	800 non-null	object
2	AccountID	800 non-null	object
3	AccountType	800 non-null	object
4	TransactionType	800 non-null	object
5	Product	800 non-null	object
6	Firm	800 non-null	object
7	Region	800 non-null	object
8	Manager	800 non-null	object
9	TransactionDate	800 non-null	object
10	TransactionAmount	800 non-null	float64
11	AccountBalance	800 non-null	float64
12	RiskScore	800 non-null	float64
13	CreditRating	800 non-null	int64
14	TenureMonths	800 non-null	int64

dtypes: float64(3), int64(3), object(9)
memory usage: 93.9+ KB
None

	TransactionID	TransactionAmount	AccountBalance	RiskScore	\
--	---------------	-------------------	----------------	-----------	---

count	800.000000	800.000000	800.000000	800.000000
mean	100.697500	54501.866711	72947.111696	0.466517
std	58.592319	29676.379125	34206.072059	0.233783
min	1.000000	-27916.911650	-35891.001750	-0.290309
25%	50.750000	34600.682150	51133.057895	0.312410
50%	98.500000	55524.078060	73006.751950	0.459326
75%	154.000000	73485.547887	94553.225110	0.630247
max	199.000000	165004.536900	171767.657800	1.200214

	CreditRating	TenureMonths
count	800.000000	800.000000
mean	580.015000	125.245000
std	154.075237	67.295139
min	306.000000	6.000000
25%	451.750000	68.000000
50%	575.000000	127.000000
75%	712.250000	183.250000
max	849.000000	239.000000

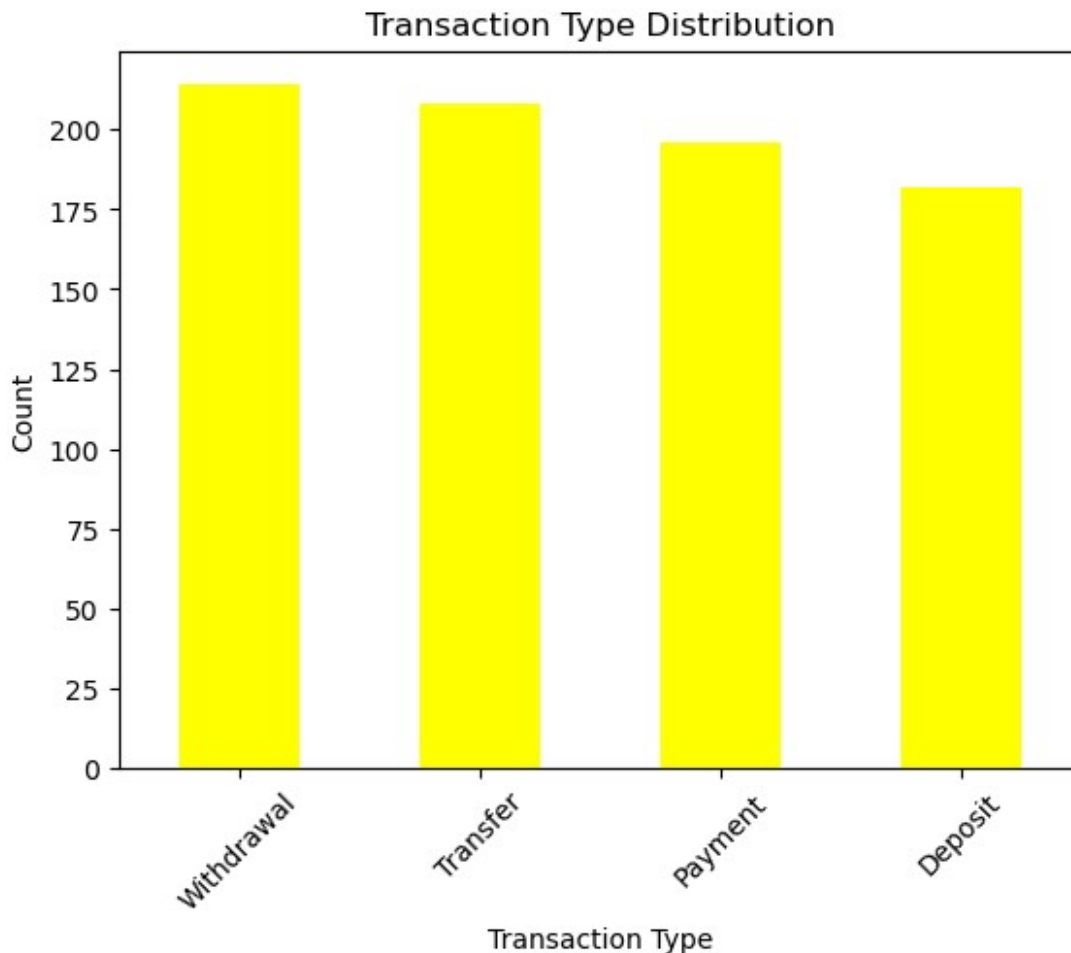
Transaction Type Distribution¶

- This code cleans the TransactionType column by converting values to text, removing extra spaces, and formatting them properly. It then counts the frequency of each transaction type using value_counts(). A bar chart is plotted to show the distribution of transaction types with labels and rotated x-axis values for better readability. Finally, df.reset_index(inplace=True) resets the DataFrame index.

```
df['TransactionType'] =
df['TransactionType'].astype(str).str.strip().str.title()

txn_counts = df['TransactionType'].value_counts()

plt.figure()
txn_counts.plot(kind='bar',color='yellow')
plt.title('Transaction Type Distribution')
plt.xlabel('Transaction Type')
plt.ylabel('Count')
plt.xticks(rotation=45)
plt.show()
df.reset_index(inplace=True)
```



Credits vs Debits Over Time

- This code converts the TransactionDate column into datetime format and sets it as the index for time-based analysis. It creates Credit and Debit columns by separating deposit amounts as credits and withdrawal, payment, or transfer amounts as debits. The data is then resampled monthly to calculate total credits and debits for each month. A line chart is plotted to compare monthly credits and debits over time. Finally, the DataFrame index is reset.

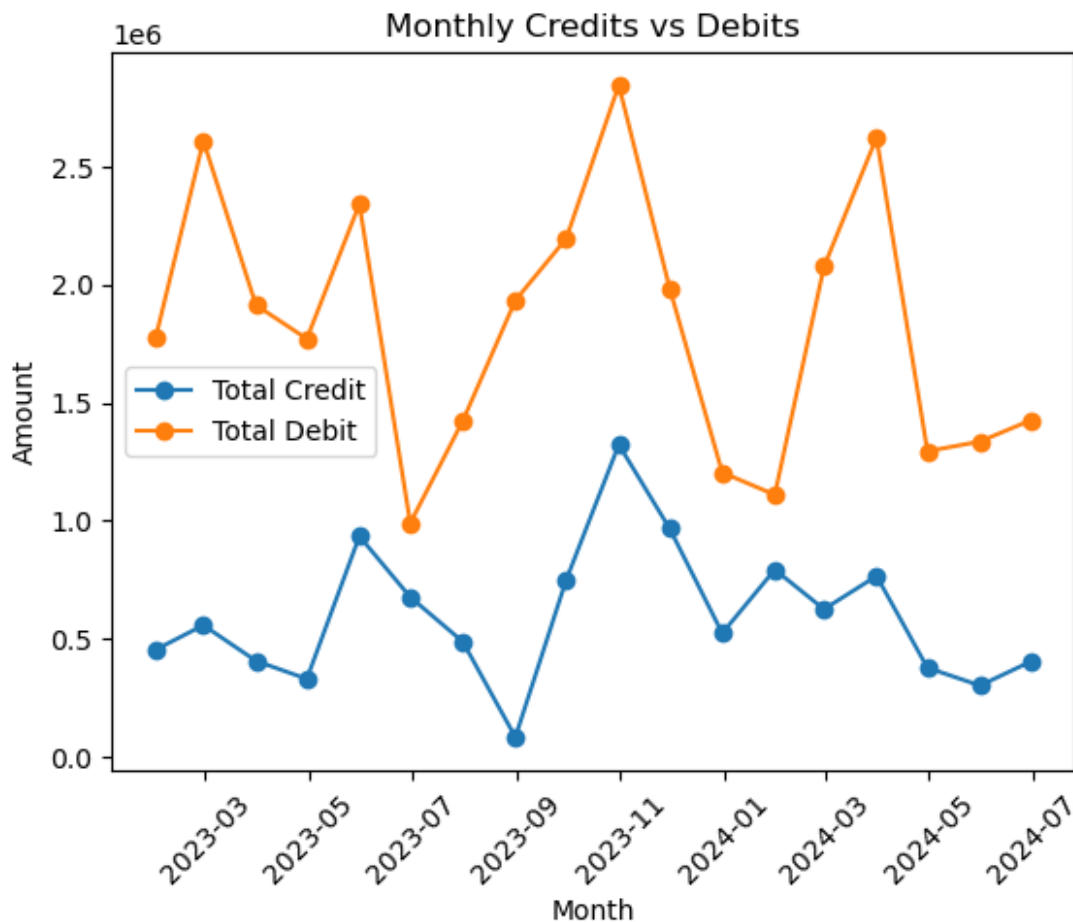
```
df.reset_index(inplace=True)

df['TransactionDate'] = pd.to_datetime(df['TransactionDate'],
dayfirst=True)

df.set_index('TransactionDate', inplace=True)

df['Credit'] = df['TransactionAmount'].where(df['TransactionType'] ==
'Deposit', 0)
df['Debit'] =
df['TransactionAmount'].where(df['TransactionType'].isin(["Withdrawal"
```

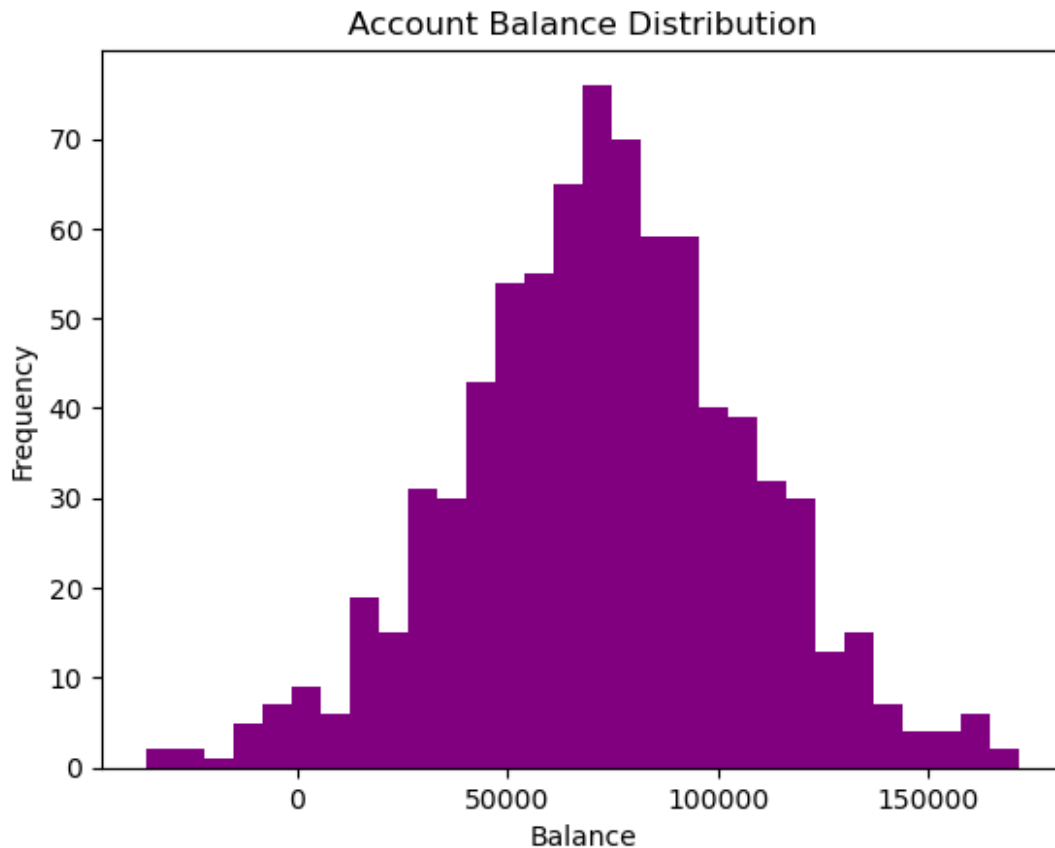
```
, "Payment", 'Transfer']], 0)
monthly_summary = df.resample('ME')[['Credit', 'Debit']].sum()
plt.figure()
plt.plot(monthly_summary.index, monthly_summary['Credit'], marker='o',
label='Total Credit')
plt.plot(monthly_summary.index, monthly_summary['Debit'], marker='o',
label='Total Debit')
plt.xlabel('Month')
plt.ylabel('Amount')
plt.title('Monthly Credits vs Debits')
plt.legend()
plt.xticks(rotation=45)
plt.show()
df.reset_index(inplace=True)
```



Account Balance Distribution

- This code creates a histogram of the AccountBalance column using 30 bins. It shows the distribution of account balances, where the x-axis represents balance amounts and the y-axis represents frequency. The chart helps visualize how account balances are spread across customers.

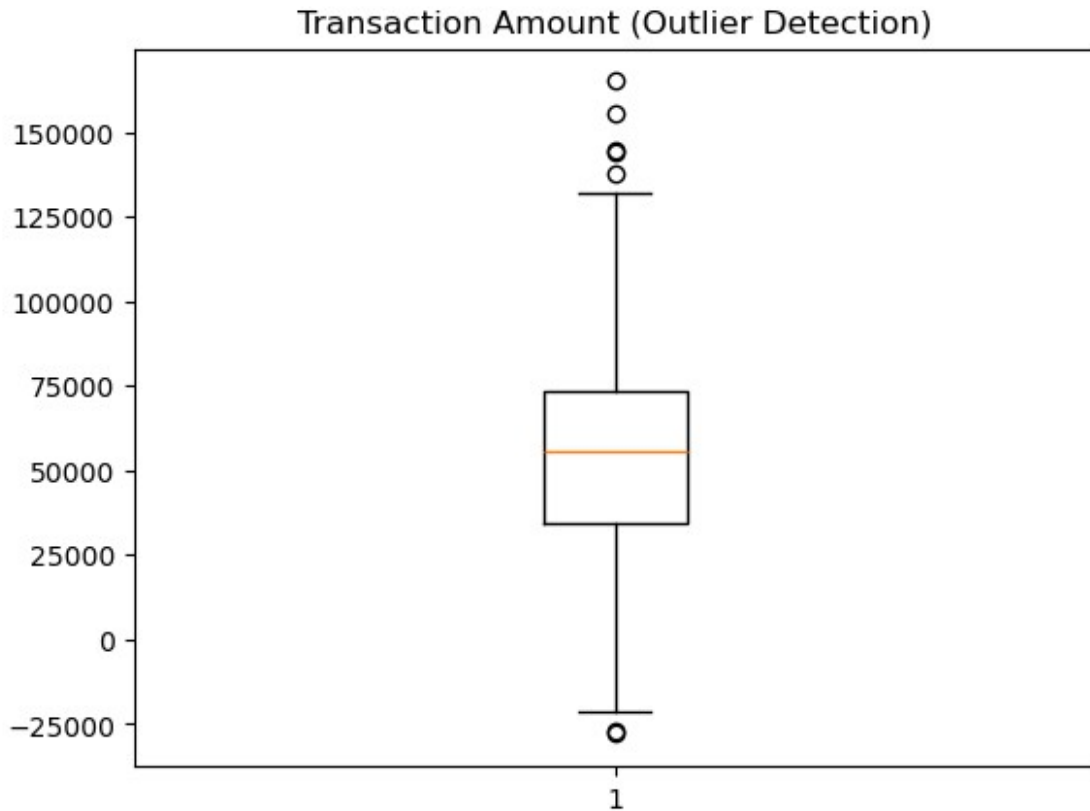
```
plt.figure()
plt.hist(df['AccountBalance'], bins=30,color='purple')
plt.title('Account Balance Distribution')
plt.xlabel('Balance')
plt.ylabel('Frequency')
plt.show()
```



Transaction Amount Distribution

- This code creates a box plot of the TransactionAmount column. It is used to visualize the spread of transaction amounts and detect outliers. The box shows the interquartile range, the center line represents the median, and points outside the whiskers indicate unusual transaction values.

```
plt.figure()
plt.boxplot(df['TransactionAmount'])
plt.title('Transaction Amount (Outlier Detection)')
plt.show()
```

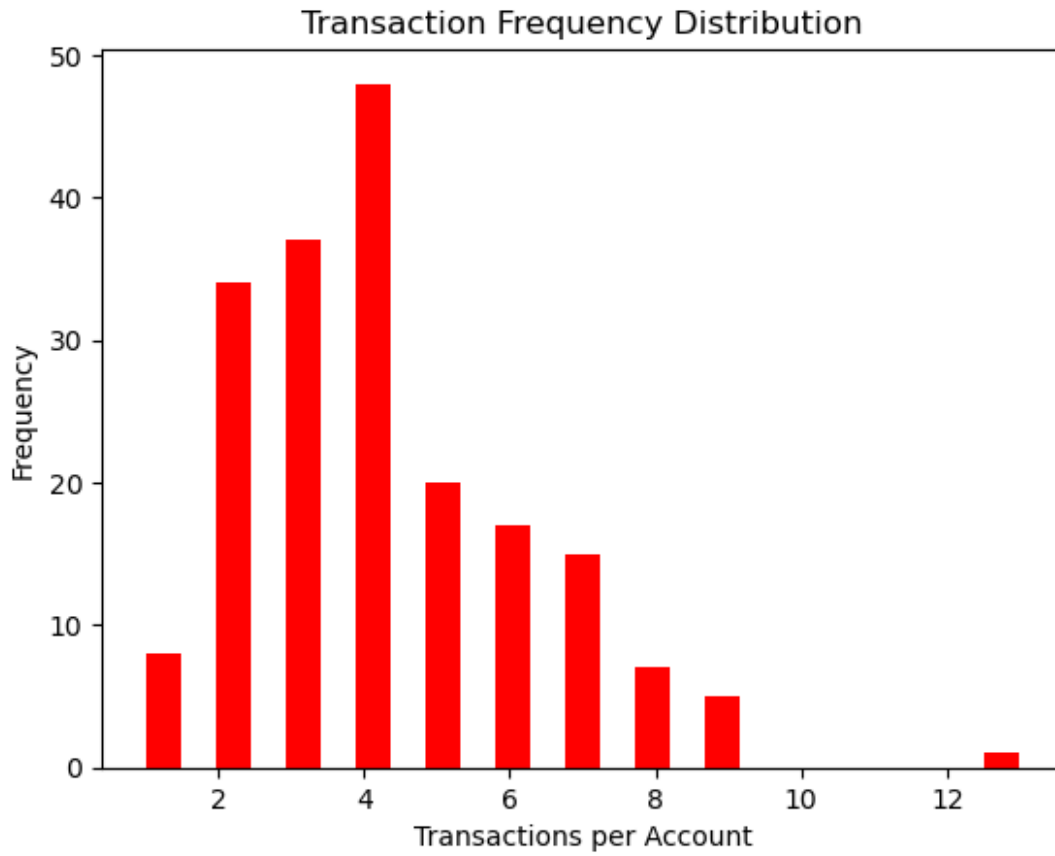


Activity Level Distribution¶

- This code groups the data by AccountID and counts the number of transactions for each account. It then plots a histogram to show the distribution of transaction frequency. The x-axis represents transactions per account, and the chart helps identify accounts with low or high activity levels.

```
txn_count = df.groupby('AccountID').size()

plt.figure()
txn_count.plot(kind='hist', bins=25,color='red')
plt.title('Transaction Frequency Distribution')
plt.xlabel('Transactions per Account')
plt.show()
```



Top Accounts by Net flow¶

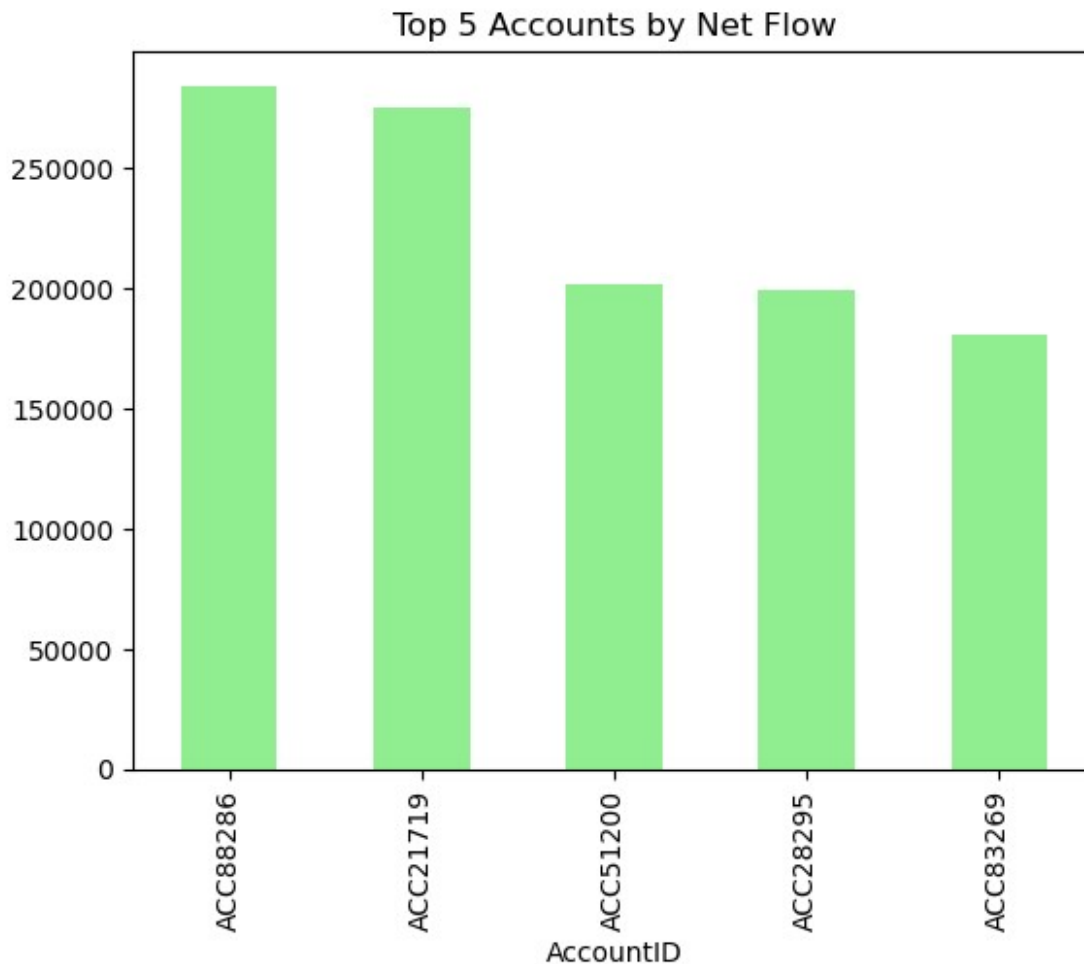
- This code calculates NetFlow by subtracting Debit from Credit for each transaction. It then groups the data by AccountID and sums the net flow for each account. The accounts are sorted in descending order, and the top 5 accounts with highest net flow are selected. Finally, a bar chart is plotted to display the top 5 accounts by net flow.

```
df['NetFlow'] = df['Credit'] - df['Debit']

account_flow = df.groupby('AccountID')
['NetFlow'].sum().sort_values(ascending=False)

top5 = account_flow.head(5)

plt.figure()
top5.plot(kind='bar', color='lightgreen')
plt.title('Top 5 Accounts by Net Flow')
plt.show()
```



6. Hypothesis Testing

6.1 Test whether high-volume transaction accounts have statistically higher average balances than low-volume accounts.¶

- This code imports `ttest_ind` from `scipy.stats` to perform an independent t-test. It first calculates transaction count and average account balance for each `AccountID`, then merges both results into one summary table. Accounts are divided into high-volume and low-volume groups based on the median transaction count. A t-test is applied to compare the average balances of both groups. Finally, the t-statistic and p-value are printed, and a conclusion is made whether there is a significant difference in balances.

```
from scipy.stats import ttest_ind
# Step 1: Transaction count per account
txn_count =
df.groupby('AccountID').size().reset_index(name='TxnCount')

# Step 2: Average balance per account
avg_balance = df.groupby('AccountID')
['AccountBalance'].mean().reset_index()
```

```

# Merge both
summary = pd.merge(txn_count, avg_balance, on='AccountID')

# Step 3: Define threshold (median)
threshold = summary['TxnCount'].median()

# Split groups
high_volume = summary[summary['TxnCount'] >= threshold]
low_volume = summary[summary['TxnCount'] < threshold]

# Step 4: Perform t-test
t_stat, p_value = ttest_ind(high_volume, low_volume)

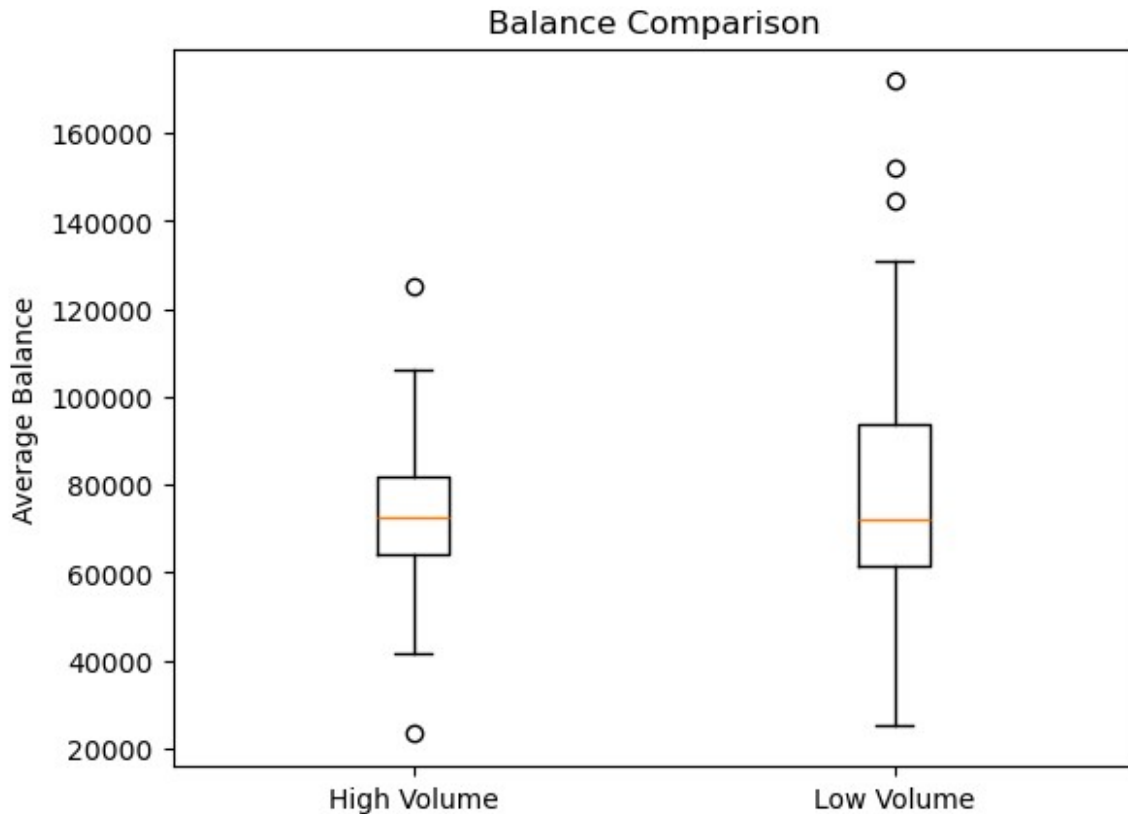
# Output
print("T-Statistic:", t_stat)
print("P-Value:", p_value)

# Conclusion
if p_value < 0.05:
    print("Reject H0 → High-volume accounts have significantly higher balances")
else:
    print("Fail to reject H0 → No significant difference")

T-Statistic: -1.6362297951660512
P-Value: 0.10344688797510922
Fail to reject H0 → No significant difference

plt.boxplot([high_volume, low_volume], tick_labels=['High Volume', 'Low Volume'])
plt.title('Balance Comparison')
plt.ylabel('Average Balance')
plt.show()

```



6.2 Conduct hypothesis testing based on segmentation.

- This code calculates transaction count and average account balance for each AccountID, then merges both results into one summary table. Accounts are segmented into High Activity and Low Activity groups based on the median transaction count. The account balances of both groups are separated, and Welch's t-test is performed to compare their mean balances. Finally, the t-statistic and p-value are displayed, and a conclusion is given on whether there is a significant difference between the two segments.

```
# Step 1: Transaction count per account
txn_count =
df.groupby('AccountID').size().reset_index(name='TxnCount')

# Step 2: Average balance per account
avg_balance = df.groupby('AccountID')
['AccountBalance'].mean().reset_index()

# Merge
summary = pd.merge(txn_count, avg_balance, on='AccountID')

# Step 3: Segmentation (based on median)
threshold = summary['TxnCount'].median()

summary['Segment'] = summary['TxnCount'].apply(
    lambda x: 'High Activity' if x >= threshold else 'Low Activity'
)
```

```

# Step 4: Split groups
high_group = summary[summary['Segment'] == 'High Activity']
['AccountBalance']
low_group = summary[summary['Segment'] == 'Low Activity']
['AccountBalance']

# Step 5: Perform t-test (Welch's recommended)
t_stat, p_value = ttest_ind(high_group, low_group, equal_var=False)

# Output
print("T-Statistic:", t_stat)
print("P-Value:", p_value)

# Conclusion
if p_value < 0.05:
    print("Reject H0 → Significant difference between segments")
else:
    print("Fail to reject H0 → No significant difference")

T-Statistic: -1.4829959732753735
P-Value: 0.14094274632649229
Fail to reject H0 → No significant difference

plt.boxplot([high_group, low_group], tick_labels=['High Activity',
'Low Activity'])
plt.title('Balance Comparison by Segment')
plt.ylabel('Average Balance')
plt.show()

```

